



MINISTÉRIO DA EDUCAÇÃO  
SECRETARIA DE EDUCAÇÃO PROFISSIONAL E TECNOLÓGICA  
INSTITUTO FEDERAL CATARINENSE - CAMPUS BLUMENAU  
CURSO DE GRADUAÇÃO EM BACHAREL EM ENGENHARIA ELÉTRICA

Arthur Pedro Ferreira

**Arquitetura de um Sistema Supervisório Orientada a Serviços para a Indústria 4.0**

Blumenau  
2026

Arthur Pedro Ferreira

## **Arquitetura de um Sistema Supervisório Orientada a Serviços para a Indústria 4.0**

Trabalho de Conclusão de Curso apresentado ao Curso de Bacharelado em Engenharia Elétrica do Instituto Federal Catarinense, Campus Blumenau, como requisito parcial para a obtenção do título de Bacharel em Engenharia Elétrica.

Orientador: Prof. Vitor Mateus Moraes

Blumenau  
2026

No meio da dificuldade, encontra-se a oportunidade.  
Albert Einstein

# Resumo

Este trabalho apresenta o desenvolvimento de um sistema supervisório web orientado a serviços, voltado ao monitoramento e à supervisão de processos industriais sobre infraestrutura em nuvem. No contexto da Indústria 4.0, a digitalização das camadas de supervisão e gestão e a adoção da arquitetura orientada a serviços (SOA) permitem ofertar, como serviços Web, funcionalidades antes restritas ao ambiente físico da planta. Os sistemas supervisórios (SCADA) convencionais, porém, exigem investimento elevado em licenças proprietárias, hardware dedicado e infraestrutura local, o que os mantém fora do alcance de pequenas e médias empresas. Diante disso, propõe-se uma solução no modelo de Software como Serviço (*Software as a Service*, SaaS), que substitui o investimento inicial por uma assinatura sob demanda e amplia o alcance do monitoramento a instalações geograficamente dispersas. A arquitetura, organizada em camadas, separa a interface web de supervisão, os serviços de domínio executados na nuvem, a persistência dos dados e a conectividade com os equipamentos de campo, cuja aquisição ocorre por meio do protocolo Modbus TCP/RTU. São revisados os conceitos de Internet das Coisas (IoT), Internet Industrial das Coisas (IIoT), sistemas ciber-físicos (CPS) e Indústria 4.0 que fundamentam a proposta, e os requisitos clássicos do SCADA, aquisição e registro de dados, sincronismo de tempo, níveis de acesso, comando e controle, escalabilidade e redundância, são mapeados sobre os componentes implementados. Ao final, espera-se validar o módulo de monitoramento por meio de testes e de um cenário demonstrativo, evidenciando o correto acesso às variáveis de processo e a estabilidade da solução frente ao volume de dados esperado.

**Palavras-chave:** SCADA; sistema supervisório web; computação em nuvem; arquitetura orientada a serviços; Indústria 4.0; Modbus.

# Abstract

This work presents the development of a service-oriented web supervisory system aimed at the monitoring and supervision of industrial processes on cloud infrastructure. Within the context of Industry 4.0, the digitalization of the supervision and management layers and the adoption of the Service-Oriented Architecture (SOA) make it possible to deliver, as Web Services, functionalities once restricted to the physical environment of the plant. Conventional supervisory systems (SCADA), however, require high investment in proprietary licenses, dedicated hardware and local infrastructure, which keeps them out of reach for small and medium-sized enterprises. In view of this, a solution based on the Software as a Service (SaaS) model is proposed, replacing the upfront investment with an on-demand subscription and extending the reach of monitoring to geographically dispersed installations. The architecture, organized in layers, separates the web supervision interface, the domain services running in the cloud, data persistence and connectivity with the field equipment, whose acquisition is carried out through the Modbus TCP/RTU protocol. The concepts of Internet of Things (IoT), Industrial Internet of Things (IIoT), cyber-physical systems (CPS) and Industry 4.0 that underpin the proposal are reviewed, and the classic SCADA requirements, data acquisition and logging, time synchronization, operation levels, command and control, scalability and redundancy, are mapped onto the implemented components. Ultimately, the monitoring module is expected to be validated through tests and a demonstrative scenario, evidencing the correct access to process variables and the stability of the solution under the expected data volume.

**Keywords:** SCADA; web supervisory system; cloud computing; service-oriented architecture; Industry 4.0; Modbus.

## Lista de Figuras

|   |                                                                                              |    |
|---|----------------------------------------------------------------------------------------------|----|
| 1 | Linha do Tempo das Revoluções Industriais . . . . .                                          | 11 |
| 2 | Pirâmide de Automação . . . . .                                                              | 12 |
| 3 | Interseções de IoT, CPS, IIoT e Indústria 4.0 . . . . .                                      | 19 |
| 4 | Arquitetura comparada: máquinas virtuais (esquerda) e contêineres Docker (direita) . . . . . | 20 |
| 5 | Aplicação monolítica (esquerda) versus arquitetura de microsserviços (direita) .             | 23 |
| 6 | Arquitetura de microsserviços baseada em barramento de eventos (broker Molecular) . . . . .  | 24 |
| 7 | Arquitetura do Sistema . . . . .                                                             | 26 |

## Lista de Siglas

| Sigla | Significado                                                                                   |
|-------|-----------------------------------------------------------------------------------------------|
| CLP   | Controlador Lógico Programável                                                                |
| CNC   | Controlador Numérico Computadorizado                                                          |
| CPS   | Sistema Ciber-Físico ( <i>Cyber-Physical System</i> )                                         |
| ERP   | <i>Enterprise Resource Planning</i> (Planejamento dos Recursos Empresariais)                  |
| I/O   | Entrada/Saída ( <i>Input/Output</i> )                                                         |
| IHM   | Interface Homem-Máquina                                                                       |
| IoT   | Internet das Coisas                                                                           |
| SCADA | Supervisão, Controle e Aquisição de Dados ( <i>Supervisory Control and Data Acquisition</i> ) |
| SOA   | Arquitetura Orientada a Serviços ( <i>Service Oriented Architecture</i> )                     |
| WS    | Serviços Web ( <i>Web Services</i> )                                                          |

## Lista de Simbolos

## Lista de Equações

## Lista de Tabelas

|   |                                          |    |
|---|------------------------------------------|----|
| 1 | Cronograma de atividades do TCC. . . . . | 32 |
|---|------------------------------------------|----|

# Índice

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução</b>                                  | <b>11</b> |
| 1.1      | Contextualização . . . . .                         | 11        |
| 1.2      | Justificativa . . . . .                            | 12        |
| 1.3      | Objetivos . . . . .                                | 13        |
| 1.3.1    | Objetivo Geral . . . . .                           | 13        |
| 1.3.2    | Objetivos Específicos . . . . .                    | 13        |
| 1.4      | Metodologia . . . . .                              | 14        |
| <b>2</b> | <b>Conceitos e Tecnologias Utilizadas</b>          | <b>15</b> |
| 2.1      | Sistema SCADA . . . . .                            | 15        |
| 2.1.1    | Aquisição e Registro de Dados . . . . .            | 15        |
| 2.1.2    | Sincronismo de Tempo . . . . .                     | 15        |
| 2.1.3    | Níveis de Acesso . . . . .                         | 16        |
| 2.1.4    | Comando e Controle . . . . .                       | 16        |
| 2.1.5    | Escalabilidade . . . . .                           | 16        |
| 2.1.6    | Redundância . . . . .                              | 16        |
| 2.2      | Automação e Tecnologias da Indústria 4.0 . . . . . | 17        |
| 2.3      | Banco de Dados . . . . .                           | 19        |
| 2.4      | Redis . . . . .                                    | 19        |
| 2.5      | Docker . . . . .                                   | 20        |
| 2.6      | Computação em Nuvem . . . . .                      | 21        |
| 2.7      | Next.js . . . . .                                  | 21        |
| 2.8      | HTTP, API REST e JSON . . . . .                    | 22        |
| 2.9      | Microserviços . . . . .                            | 23        |
| 2.10     | Molecular . . . . .                                | 23        |
| 2.11     | Protocolo Modbus TCP e Modbus RTU . . . . .        | 24        |
| <b>3</b> | <b>Funcionamento</b>                               | <b>26</b> |
| 3.1      | Validações dos Requisitos do SCADA . . . . .       | 26        |
| 3.1.1    | Aquisição e Registro de Dados . . . . .            | 27        |
| 3.1.2    | Sincronismo de Tempo . . . . .                     | 27        |
| 3.1.3    | Níveis de Acesso . . . . .                         | 27        |
| 3.1.4    | Comando e Controle . . . . .                       | 27        |
| 3.1.5    | Escalabilidade . . . . .                           | 28        |
| 3.1.6    | Redundância . . . . .                              | 28        |
| 3.2      | Servidores . . . . .                               | 28        |
| 3.2.1    | Aplicação Next.js (Frontend e Backend) . . . . .   | 29        |
| 3.2.2    | Banco de Dados . . . . .                           | 29        |
| 3.2.3    | Cache . . . . .                                    | 30        |
| 3.2.4    | BullMQ . . . . .                                   | 30        |
| 3.2.5    | Barramento Molecular . . . . .                     | 30        |
| 3.2.6    | Workers de Aquisição . . . . .                     | 30        |
| 3.3      | Aquisição de Dados . . . . .                       | 31        |
| <b>4</b> | <b>Cronograma</b>                                  | <b>32</b> |

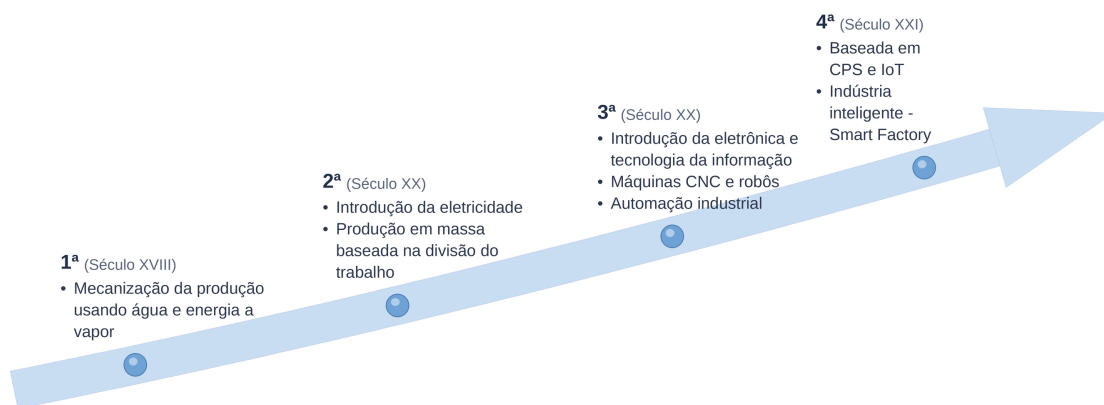
# 1 Introdução

## 1.1 Contextualização

A indústria é o setor da economia dedicado à produção de bens materiais, apoiado em trabalho humano, energia e equipamentos de alto valor agregado. Ao longo de sua evolução, sucessivos saltos tecnológicos redefiniram processos, produtos e formas de organização do trabalho, as chamadas "revoluções industriais" (Lasi *et al.*, 2014).

A Figura 1 resume essa linha do tempo. Na primeira fase, destaca-se a mecanização da produção, na segunda, a difusão da energia elétrica e a produção em massa baseada no modelo Fordista e na terceira fase, a digitalização (Lasi *et al.*, 2014). Foi nessa fase que a introdução da tecnologia da informação, da automação e da eletrônica impulsionou o início da automação dos processos produtivos. Surgiram então os controladores lógicos programáveis (CLPs), os Controladores Numéricos Computadorizados (CNCs) e os robôs (Pisching, 2017). Essas tecnologias tornaram-se essenciais para as indústrias que buscam maior eficiência e qualidade com menores custos e prazos. Sua adoção contribui diretamente para o aumento da produtividade (Bigheti, 2020).

Figura 1: Linha do Tempo das Revoluções Industriais



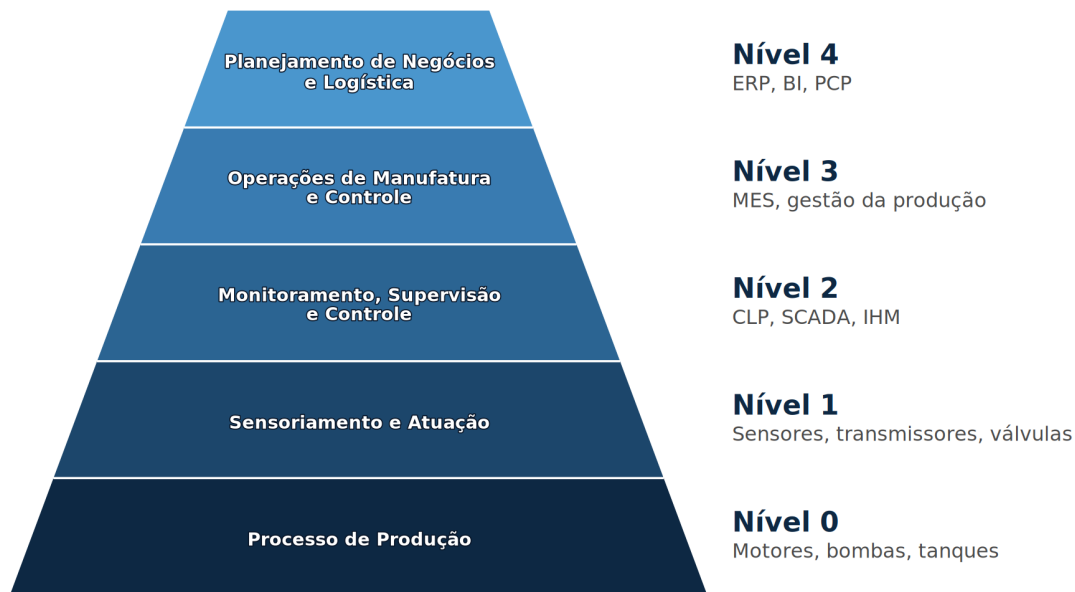
Fonte: Adaptado de Pisching (2017).

Atualmente, vive-se a quarta fase, a chamada Indústria 4.0, baseada na indústria inteligente, na Internet das Coisas (*Internet of Things*, IoT) e nos sistemas ciber-físicos (*Cyber-Physical Systems*, CPS) (Bigheti, 2020). Nesses sistemas, subsistemas computacionais cooperam entre si e permanecem integrados ao mundo físico e a seus processos, o que permite medir, controlar e analisar dados em tempo real por meio da rede, inclusive a Internet (Monostori *et al.*, 2016).

A Figura 2 organiza a automação na Indústria 4.0 em forma de pirâmide, distribuída em cinco níveis. No nível 0 está o processo físico de produção, com elementos como motores, bombas e tanques, no nível 1, o sensoriamento e a atuação, realizados por sensores, transmissores e válvulas. O nível 2 reúne o monitoramento, a supervisão e o controle, camada em que se encontram os controladores lógicos programáveis (CLPs), os sistemas supervisórios (SCADA) e as interfaces homem-máquina (IHM), responsáveis por acompanhar e comandar o processo em tempo real e por se comunicar com o nível 1 por protocolos seriais ou diretamente pelas entradas e saídas (I/O). O nível 3 corresponde às operações de manufatura e ao controle da produção, em que atuam os sistemas de execução da manufatura (*Manufacturing Execution Systems*, MES). No topo, o nível 4 abrange o planejamento de negócios e a logística, em que

se destacam os sistemas de gestão empresarial (*Enterprise Resource Planning*, ERP) (Bigheti, 2020; Bangemann *et al.*, 2014).

Figura 2: Pirâmide de Automação



Fonte: Adaptado de Bangemann *et al.* (2014).

Recentemente, as camadas de supervisão e de gestão, do nível 2 ao nível 4, ganharam mais notoriedade: ao digitalizar serviços antes restritos ao ambiente físico da planta, passou-se a ofertá-los sobre infraestrutura em nuvem e a compartilhá-los entre diferentes sistemas e usuários. Em geral, esses serviços seguem a arquitetura orientada a serviços (*Service Oriented Architecture*, SOA), materializada por serviços Web (*Web Services*, WS) (Bangemann *et al.*, 2014; Abbas, 2011). Esse movimento é sustentado pela própria natureza da Indústria 4.0, que se caracteriza pelo grande avanço dos protocolos de comunicação e de telemetria, pela popularização do desenvolvimento de software e pela interconexão inteligente de máquinas e sistemas (Lasi *et al.*, 2014; Pisching, 2017; Bigheti, 2020).

Tais avanços permitiram o monitoramento e a supervisão centralizada de processos complexos em tempo real, elevando o papel dos sistemas supervisórios de simples interfaces de operação para centrais estratégicas de análise de dados. Nesse cenário, este trabalho propõe o desenvolvimento de um sistema supervisório web orientado a serviços, que disponibiliza sobre infraestrutura em nuvem funcionalidades antes restritas ao ambiente físico da planta (Bangemann *et al.*, 2014). Alinhada às tendências da Indústria 4.0 (Bigheti, 2020), a proposta torna o monitoramento e a supervisão acessíveis e compartilháveis entre diferentes sistemas e usuários.

## 1.2 Justificativa

Os processos industriais geram grande volume de dados, utilizados para acompanhar variáveis e interpretar a tendência do processo. Em geral, essa informação é organizada e apresentada ao operador por um software de supervisão, por meio de uma interface homem-máquina (IHM), que converte leituras de campo em telas, tendências e alarmes de forma legível e acionável (Zanghi, 2019). Com o aumento da capacidade de processamento dos computadores,

o aprimoramento dos protocolos de comunicação e a maior integração com redes externas, inclusive a Internet, consolidou-se a supervisão remota e cresceu a exigência de visibilidade em tempo real sobre o que ocorre na planta industrial. Em setores como saneamento, transporte e energia, nos quais instalações costumam estar geograficamente espalhadas, ganham peso a telemetria e o controle supervisionado à distância. De modo esquemático, uma arquitetura típica de sistema SCADA integra os equipamentos de campo, como CLPs, aos módulos de monitoramento e controle e centraliza a visualização dos dados para o operador (Abbas, 2011).

Apesar de consolidados, os sistemas supervisórios convencionais costumam exigir investimento elevado em licenças de software proprietário, hardware dedicado e infraestrutura local, o que os mantém fora do alcance de pequenas e médias empresas, para as quais a aquisição de uma licença raramente se justifica diante do porte da operação (Phuyal *et al.*, 2020). Nesse cenário, oferecer a supervisão como um serviço web, substitui esse investimento inicial por uma assinatura sob demanda, alinhada ao uso efetivo do sistema, e democratiza o acesso a uma tecnologia antes economicamente inviável para esse público. Além de reduzir custos, a supervisão via web amplia o alcance do monitoramento e do controle a instalações geograficamente dispersas e favorece a colaboração entre equipes em locais distintos, reduzindo deslocamentos presenciais e agilizando a resposta a ocorrências (Abbas, 2011).

A computação em nuvem é um modelo em que recursos computacionais, como armazenamento, processamento, bancos de dados e aplicações, são ofertados como serviços remotos, flexíveis e escaláveis (Pedrosa; Nogueira, 2011). Em geral, o usuário não precisa conhecer a localização física nem os detalhes da infraestrutura que sustenta esses serviços, o consumo costuma ser sob demanda, alinhado ao uso, o que reduz a necessidade de investimento direto em hardware e software no ambiente local (Henriques da Silva, 2010). Por sua vez, a conectividade, em especial por meio de redes sem fio, é essencial para que dispositivos e sistemas ciber-físicos mantenham troca de dados com a nuvem e com demais elementos da arquitetura distribuída (Stankovic, 2014; Monostori *et al.*, 2016).

## 1.3 Objetivos

A seguir são apresentados os objetivos gerais e específicos deste Trabalho de Conclusão de Curso.

### 1.3.1 Objetivo Geral

Desenvolver um sistema supervisório web com arquitetura orientada a serviços, capaz de apoiar o monitoramento e a supervisão de processos industriais, com serviços bem definidos, reutilizáveis e acessíveis via interfaces web.

### 1.3.2 Objetivos Específicos

- Revisar os conceitos de Indústria 4.0, monitoramento industrial, arquitetura orientada a serviços (SOA) e serviços Web, de modo a embasar a solução proposta.
- Especificar os requisitos funcionais e não funcionais e a arquitetura lógica do sistema, com destaque para a separação entre camada de apresentação (web), camada de serviços e camada de obtenção ou persistência dos dados de monitoramento.
- Implementar os serviços de backend responsáveis pela leitura periódica ou sob demanda, pelo registro e pela disponibilização dos dados de processo de forma modular e interoperável.

- Implementar a interface web de monitoramento, permitindo a visualização dos principais indicadores, tendências ou telas resumo associadas ao processo acompanhado.
- Validar o módulo de monitoramento por meio de testes e de um cenário demonstrativo, evidenciando o correto acesso às variáveis e a estabilidade da solução frente ao volume de dados esperado.

## 1.4 Metodologia

O desenvolvimento deste trabalho é dividido em etapas, com o objetivo de estruturar, implementar e validar um sistema supervisão web com arquitetura orientada a serviços, voltado ao monitoramento e à supervisão de processos industriais. Cada etapa parte dos resultados da anterior, de modo que a fundamentação teórica orienta a especificação, que por sua vez direciona a implementação e, por fim, a validação da solução proposta. Além deste capítulo de introdução, o trabalho está organizado em outros cinco capítulos, associados às etapas descritas a seguir.

Inicialmente, é realizada uma revisão bibliográfica sobre os conceitos que sustentam o trabalho, abrangendo automação e informática industrial, monitoramento e sistemas supervisórios (SCADA), Indústria 4.0 e suas tecnologias correlatas, Internet das Coisas (*Internet of Things*, IoT) e sistemas ciberfísicos (*Cyber-Physical Systems*, CPS). Nessa etapa também são estudados a arquitetura orientada a serviços (*Service-Oriented Architecture*, SOA), os serviços Web (*Web Services*), a computação em nuvem (*cloud computing*) e o protocolo Modbus em suas variantes TCP e RTU, com foco nos requisitos de aquisição, registro e disponibilização dos dados de processo. Esses conceitos e tecnologias, bem como a forma com que se relacionam para a solução proposta, são apresentados no Capítulo 2.

Na etapa seguinte, são especificados os requisitos funcionais e não funcionais e a arquitetura lógica do sistema, com destaque para a separação entre a camada de apresentação (web), a camada de serviços e a camada de obtenção ou persistência dos dados de monitoramento. A definição dessa arquitetura em camadas estabelece as fronteiras entre os componentes e orienta as decisões de implementação adotadas nas etapas posteriores, sendo detalhada no Capítulo 3.

Posteriormente, são implementados os serviços de *backend* responsáveis pela leitura periódica ou sob demanda, pelo registro e pela disponibilização dos dados de processo de forma modular e interoperável. Nessa etapa, são desenvolvidos os mecanismos de aquisição via Modbus TCP e RTU, o agendamento das leituras e a organização dos serviços segundo o padrão de microsserviços, de modo a garantir escalabilidade e reuso, conforme descrito no Capítulo 4.

Em seguida, é implementada a interface web de monitoramento, permitindo a visualização dos principais indicadores, tendências ou telas resumo associadas ao processo acompanhado. Essa camada consome os serviços disponibilizados pelo *backend*, evidenciando a interoperabilidade entre apresentação e serviços prevista na arquitetura, e é apresentada no Capítulo 5.

Por fim, o módulo de monitoramento é validado por meio de testes e de um cenário demonstrativo, ou estudo de caso, evidenciando o correto acesso às variáveis de processo e a estabilidade da solução frente ao volume de dados esperado. Os resultados obtidos são confrontados com os requisitos definidos na etapa de especificação, permitindo avaliar as contribuições, as limitações e as potencialidades da solução proposta, conforme o Capítulo 6.

## 2 Conceitos e Tecnologias Utilizadas

Este capítulo aborda os conceitos e as tecnologias que sustentam o trabalho: automação industrial, tecnologia da informação aplicada ao contexto industrial e desenvolvimento de software. O objetivo é reunir definições, enquadramentos e ferramentas necessários para compreender, nas seções seguintes, a proposta de sistema supervisório web orientado a serviços, explicitando como cada tema se articula com a solução adotada.

### 2.1 Sistema SCADA

A indústria evoluiu muito na busca por mais monitoramento e controle dos processos. Com o tempo, os custos de computadores, hubs e conversores de rede caíram bastante, ao passo que o desenvolvimento de software ganhava relevância. Nesse contexto, as empresas de software industrial passaram a oferecer soluções personalizadas para cada tipo de processo e indústria, com ampla gama de protocolos de comunicação. A esse tipo de sistema deu-se o nome de SCADA, sigla em inglês para *Supervisory Control and Data Acquisition* (Zanghi, 2019). De forma geral, define-se o SCADA como um pacote de software posicionado sobre o hardware ao qual se interliga, que viabiliza a supervisão e o controle de processos industriais de forma local ou remota (Phuyal *et al.*, 2020; Abbas, 2011).

Para Zanghi (2019), um sistema SCADA deve contemplar seis módulos, descritos nas subseções a seguir: aquisição e registro de dados, sincronismo de tempo, níveis de acesso, comando e controle, escalabilidade e redundância.

#### 2.1.1 Aquisição e Registro de Dados

A aquisição das informações de campo é o primeiro grande desafio em um sistema SCADA (Zanghi, 2019). Atualmente, predominam dispositivos genéricos inteligentes que disponibilizam algum protocolo de comunicação, responsáveis por adquirir grandezas analógicas e digitais e disponibilizá-las ao supervisório em formato binário por meio de uma rede de dados (Yadav; Paul, 2020). O meio de comunicação varia conforme a distância e a infraestrutura disponível, indo das redes locais cabeadas, tipicamente Ethernet, a enlaces sem fio como rádio frequência (RF) e telefonia móvel (4G) (Stouffer *et al.*, 2015), até soluções orientadas à Internet das Coisas, que trafegam os dados pela internet (Khalid *et al.*, 2024).

Os pontos aquisitados em um sistema SCADA são denominados *tags* e armazenados no banco de dados do supervisório. Cada *tag* assume um tipo de dado conforme a natureza da grandeza que representa, podendo ser booleano, inteiro, real ou texto. O registro contínuo desses valores pelo supervisório permite análises posteriores, na forma de gráficos históricos de tendência e de sequenciais de eventos (Zanghi, 2019).

#### 2.1.2 Sincronismo de Tempo

Outro requisito fundamental dos sistemas SCADA é o sincronismo de tempo. Todo dado coletado deve possuir um registro de tempo de origem, a chamada estampa de tempo (*time stamp*), de modo que cada variação de estado fique associada a um instante de referência. Essa exigência decorre da necessidade de análises posteriores de ocorrências e faltas, que dependem da cronologia precisa dos acontecimentos para permitir a identificação das causas (Zanghi, 2019).

Na prática, a aquisição é realizada de forma cíclica: o supervisório consulta periodicamente os dispositivos de campo, processo conhecido como *polling* (Stouffer *et al.*, 2015), em ciclos de

leitura sucessivos cuja frequência é definida conforme a dinâmica do processo (Yadav; Paul, 2020). Como consequência, a estampa de tempo atribuída a cada ponto corresponde ao ciclo em que o valor foi lido, e não ao instante físico exato da transição, de modo que a resolução temporal do registro fica limitada ao período do ciclo de aquisição.

### **2.1.3 Níveis de Acesso**

Os níveis de acesso de um sistema SCADA definem a separação das permissões de supervisão e controle entre os usuários. Classicamente organizada em níveis por instalação (Zanghi, 2019), essa separação materializa-se como acessos definidos por planta, em que cada usuário recebe permissões apenas sobre os pontos e comandos da instalação à qual está vinculado, segundo o controle de acesso baseado em cargos e o princípio do menor privilégio, apoiado em mecanismos adequados de autenticação e autorização (Stouffer *et al.*, 2015).

### **2.1.4 Comando e Controle**

As ações executadas remotamente pelo supervisório consistem no envio de comandos aos dispositivos de campo, isto é, na escrita de uma informação no dispositivo (Yadav; Paul, 2020). Quanto ao momento de execução, o comando pode ser instantâneo, escrevendo de imediato o valor desejado ou agendado, programado para ações conforme cenários previamente definidos, com o instante de execução registrado (Zanghi, 2019).

O processamento de um comando segue quatro fases: seleção, em que o operador escolhe na interface o equipamento; verificação de intertravamentos e condições de segurança; confirmação explícita pelo operador; e execução, com o envio efetivo da ordem ao elemento final de controle (Zanghi, 2019). Após a execução, o supervisório verifica a confirmação para garantir que a solicitação foi aceita pelo dispositivo e, em caso de falha, gera um alarme, pois comandos indevidos podem danificar equipamentos ou comprometer a segurança do processo (Stouffer *et al.*, 2015).

### **2.1.5 Escalabilidade**

A expansão das instalações é frequente em ambientes industriais, com a inclusão de novos equipamentos, pontos de medição e circuitos. Para acompanhá-la, o software SCADA deve ser escalável, permitindo acrescentar pontos à base de dados e criar novas telas de operação sem comprometer o que já está em produção (Zanghi, 2019).

A análise da escalabilidade deve abranger não apenas o supervisório, mas também os elementos de aquisição, controle e rede, que precisam permitir o acréscimo de novos dispositivos ao barramento de comunicação, integração a ser prevista já na fase inicial do projeto (Zanghi, 2019).

### **2.1.6 Redundância**

A redundância consiste no acréscimo de um elemento idêntico ao original, de modo que, em caso de falha do primeiro, o segundo assuma suas funções da forma mais transparente possível para a operação (Zanghi, 2019). A prática aplica-se a diversos componentes do sistema, como softwares supervisórios, dispositivos de aquisição e controle e elementos de rede, apesar do custo adicional, eleva a confiabilidade a graus compatíveis com a criticidade de aplicações industriais sensíveis (Salvador, 2019a).

Em uma arquitetura redundante típica, múltiplas estações de operação comunicam-se simultaneamente com os dispositivos de campo e as unidades de aquisição e controle operam em

pares espelhados (Zanghi, 2019). De modo geral, recomenda-se que cada componente crítico tenha um par redundante, sobre redes igualmente redundantes, e que o sistema seja projetado para degradação suave, preservando a supervisão e o controle mesmo diante de falhas pontuais (Stouffer *et al.*, 2015).

## 2.2 Automação e Tecnologias da Indústria 4.0

A automação industrial é o conjunto de tecnologias e práticas que substituem a intervenção humana direta na execução de processos produtivos por sistemas eletrônicos, mecânicos e computacionais capazes de medir, controlar, comandar e supervisionar variáveis físicas. A informática industrial, por sua vez, designa a aplicação das tecnologias da informação e da comunicação a esse contexto, articulando o mundo da Tecnologia da Automação (TA), voltado a tempo real determinístico, robustez e segurança operacional, com o da Tecnologia da Informação (TI), orientado a interoperabilidade, escala e segurança da informação. Bangemann *et al.* (2014) apontam a convergência entre TA e TI como um dos principais vetores de transformação da automação contemporânea, frequentemente descrita como o desfecho que conduz à Indústria 4.0 (Pisching, 2017; Bigheti, 2020).

A Internet das Coisas (*Internet of Things*, IoT), termo cunhado por Kevin Ashton em 1999, descreve uma infraestrutura de rede global na qual objetos físicos e virtuais possuem identidade e representação digital e se integram à internet por interfaces inteligentes, trocando informações de forma autônoma (Pisching, 2017; Bigheti, 2020). Stankovic (2014) a descreve como ponto de convergência entre computação móvel e pervasiva, redes de sensores sem fio e sistemas ciber-físicos, antevendo um cenário em que o sensoriamento ubíquo se torna uma utilidade comparável à eletricidade. Essa escala, da ordem de bilhões a trilhões de dispositivos, impõe desafios de nomeação, autenticação, confiabilidade, armazenamento e privacidade, agravados pelo fato de que dispositivos que atuam sobre o mundo físico tornam falhas uma questão de segurança operacional (Stankovic, 2014).

Quando esse paradigma é trazido para o domínio industrial, sensoriamento de processos, controle de máquinas e otimização de cadeias produtivas, recebe o nome de Internet Industrial das Coisas (*Industrial Internet of Things*, IIoT), um subconjunto da IoT especializado para ampliar a eficiência e a responsividade das plantas (Bigheti, 2020). Embora compartilhem o mesmo núcleo, a IoT de consumo e a IIoT diferem em criticidade: a primeira lida com cenários ad-hoc e requisitos brandos, enquanto a segunda envolve nós fixos, altos volumes de dados e exigências rigorosas de tempo real, confiabilidade e segurança (Bigheti, 2020). Para Pisching (2017), a proliferação da IoT na indústria leva fabricantes a embarcar tecnologias que sustentam a convergência TA/TI; e, à medida que dispositivos de campo expõem suas funções como serviços de rede, por exemplo via OPC-UA, a pirâmide ISA-95 tende a achatar-se em uma arquitetura logicamente plana baseada em serviços (Bangemann *et al.*, 2014).

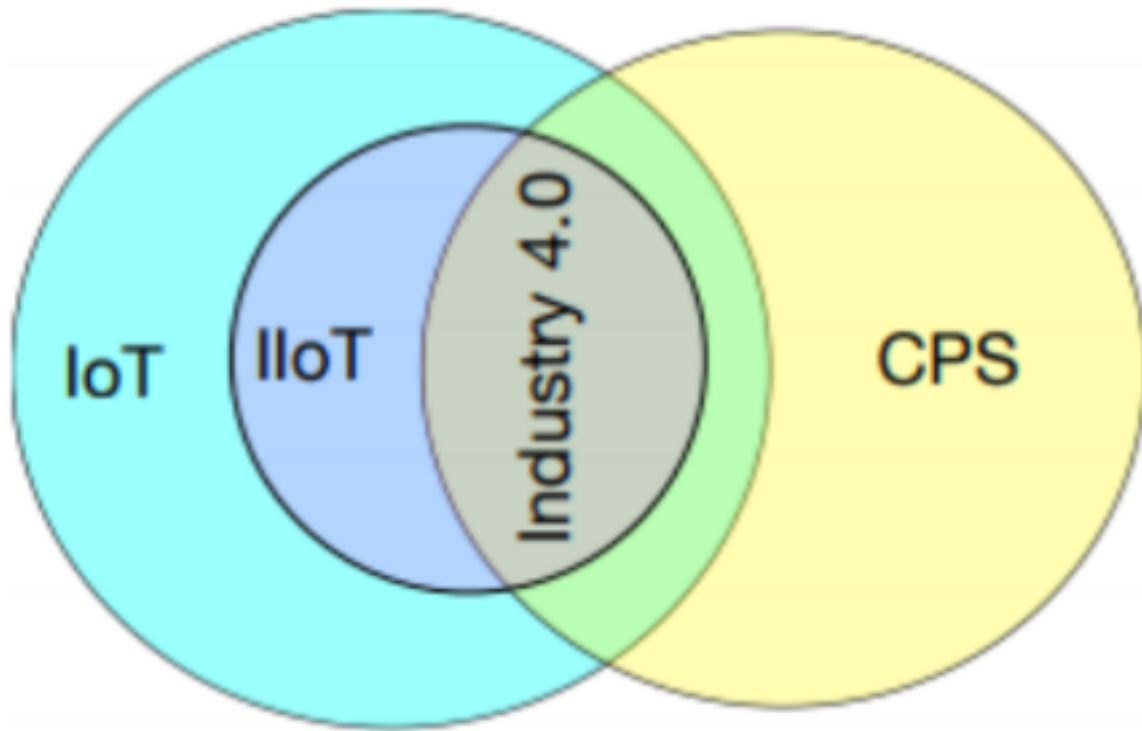
A base de comunicação que sustenta esses conceitos é a troca direta de informações entre dispositivos, denominada comunicação Máquina a Máquina (*Machine-to-Machine*, M2M). Originada na telemetria industrial e nos primeiros sistemas SCADA, a M2M precede e fundamenta a IoT e viabiliza a noção de máquina inteligente: dispositivos que coordenam ações, ajustam parâmetros e reportam estados sem intervenção manual (Bigheti, 2020; Pisching, 2017). Tecnicamente, apoia-se em uma família de protocolos, do Modbus, mais difundido em telemetria, ao MQTT, leve e do tipo publicador-assinante, e ao OPC-UA, que oferece um modelo unificado de informação para comunicação segura entre dispositivos heterogêneos (Bangemann *et al.*, 2014). Em redes de longa distância, CLPs em campo comunicam-se por meio de modems celulares com *gateways* que concentram sessões e roteiam requisições até os serviços em nu-

vem, permitindo supervisionar centenas de pontos remotos sem hardware proprietário em cada instalação.

Os Sistemas Ciber-Físicos (*Cyber-Physical Systems*, CPS), termo proposto em 2006 nos Estados Unidos, descrevem o acoplamento intensivo entre entidades computacionais e o mundo físico (Pisching, 2017; Monostori *et al.*, 2016). Mais relevante que a soma das partes é a interseção entre os mundos físico e cibernético: é a interação entre eles que caracteriza o CPS (Monostori *et al.*, 2016). Herdeiros dos sistemas embarcados, os CPS acrescentam conectividade, autonomia e cooperação, e no domínio da manufatura recebem o nome de sistemas ciber-físicos de produção. Monostori *et al.* (2016) destacam três características essenciais: a inteligência, capacidade de adquirir informações do entorno e agir autonomamente; a conectividade, uso de conexões com outros elementos, pessoas e serviços; e a responsividade, reação em tempo apropriado a mudanças. Como observa Pisching (2017), os CPS oferecem a base física e computacional sobre a qual a IoT viabiliza a Indústria 4.0.

O termo *Industrie 4.0* foi apresentado em 2011, na Feira de Hannover, e oficializado em 2013 pela estratégia de alta tecnologia do governo alemão, anunciando *ex-ante* uma quarta revolução industrial (Pisching, 2017; Bigheti, 2020). Lasi *et al.* (2014) a caracterizam pela conjunção de um *application-pull*, com demanda por personalização, flexibilidade e descentralização, e um *technology-push*, fundado na automação crescente, na digitalização e na miniaturização do *hardware*. Bigheti (2020) a define como um conjunto de tecnologias coordenadas para conferir competitividade e customização, habilitadas por CPS, IIoT, computação em nuvem, redes sem fio e uso massivo de dados, cujo conceito-síntese é a fábrica inteligente. Seus princípios de projeto, segundo Pisching (2017), são a interoperabilidade, a virtualização, a descentralização, a operação em tempo real, a orientação a serviços e a modularidade, padronizados pelo modelo de referência RAMI 4.0. A Figura 3 sintetiza as relações entre esses conceitos: a IoT como universo mais amplo, a IIoT em sua interseção com o domínio industrial, os CPS como categoria de forte acoplamento computação-físico, e a Indústria 4.0 emergindo na interseção entre IIoT e CPS. Nesse contexto, o SCADA deixa de ser um sistema fechado de supervisão local e passa a um nó de uma rede de serviços que integra produção, manutenção, logística e clientes em uma cadeia de valor digital (Bigheti, 2020; Bangemann *et al.*, 2014).

Figura 3: Interseções de IoT, CPS, IIoT e Indústria 4.0



Fonte: Bigheti (2020, p. 27).

## 2.3 Banco de Dados

O banco de dados é o componente de software responsável por armazenar, organizar e recuperar informações de forma persistente. Diferentes modelos atendem a propósitos distintos: os bancos relacionais favorecem dados estruturados e as relações entre entidades; os de séries temporais, o armazenamento massivo de medições com carimbo de tempo; e os NoSQL (documentais, chave-valor, grafos), cenários de alta variabilidade de esquema.

O modelo relacional, proposto por Edgar F. Codd em 1970 e materializado em bancos como PostgreSQL, MySQL e Oracle, organiza os dados em tabelas de linhas e colunas, cada uma definida por um esquema, uma chave primária que identifica unicamente cada registro e, opcionalmente, chaves estrangeiras que estabelecem relações entre tabelas, garantindo integridade referencial. A linguagem de consulta padrão é o SQL (*Structured Query Language*), com operações de seleção, inserção, atualização e remoção, além de junções, agregações, transações e índices. Um exemplo amplamente adotado é o *PostgreSQL*, banco relacional de código aberto com forte suporte a tipos complexos (JSON, *arrays*), consultas em janelas de tempo e extensões para séries temporais (PostgreSQL Global Development Group, 2026).

Em ambientes de produção, boas práticas incluem indexar os campos mais consultados, particionar as tabelas históricas por intervalo de tempo para evitar tabelas monolíticas e empregar réplicas com *failover* automático e *snapshots* para alta disponibilidade (Damin, 2022; Salvador, 2019a).

## 2.4 Redis

O *Redis* é um armazenamento de dados em memória, de código aberto, que opera como banco chave-valor de altíssimo desempenho. Diferentemente dos bancos relacionais, em que a

persistência em disco é a prioridade, o Redis mantém todo o conjunto de trabalho em memória RAM, oferecendo latências sub-milissegundo para operações de leitura e escrita (Redis Ltd., 2026).

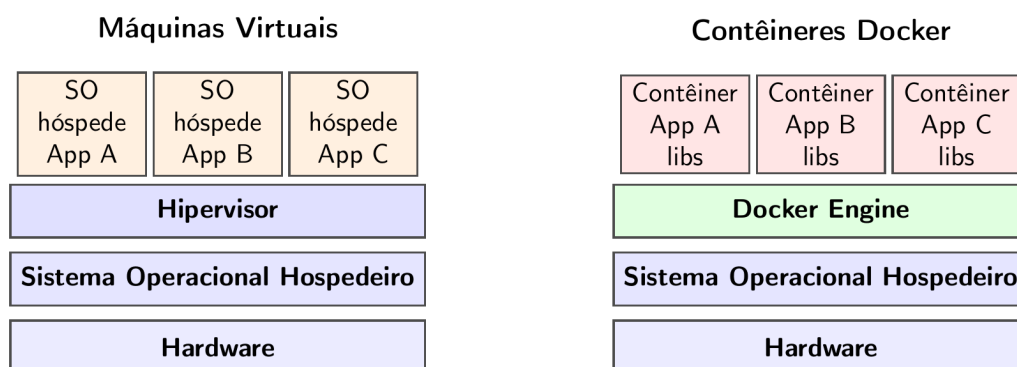
O Redis oferece persistência opcional por *snapshots* periódicos e log de operações de escrita, expiração automática de chaves por *time-to-live* (TTL) e um mecanismo publicador-assinante (Pub/Sub) para troca de mensagens entre processos. Por essas características, é comumente empregado como cache de acesso rápido, como barramento de eventos e como armazenamento de dados temporários (Redis Ltd., 2026).

Como o Redis guarda os dados em memória, ele usa cópias e recuperação automática para não parar diante de falhas. A replicação mantém uma ou mais réplicas sempre atualizadas da instância principal. Sobre isso, o *Redis Sentinel* cuida da disponibilidade: vigia a instância principal e, se ela cai, promove sozinho uma réplica ao seu lugar, sem intervenção manual. Já o *Redis Cluster* cuida do crescimento: reparte as chaves entre vários nós, de modo que basta acrescentar nós para aumentar a capacidade, e cada nó pode ter réplicas para o serviço continuar mesmo se um deles falhar (Redis Ltd., 2026).

## 2.5 Docker

O Docker é uma plataforma de virtualização baseada em contêineres, unidades de empacotamento que reúnem código, bibliotecas e dependências em um artefato portátil (Docker, Inc., 2026). Diferentemente das máquinas virtuais (Salvador, 2019b), que emulam uma máquina inteira com seu sistema operacional, os contêineres compartilham o *kernel* do hospedeiro e isolam apenas o espaço de processos e o sistema de arquivos da aplicação (Figura 4). Por isso são ordens de magnitude mais leves: enquanto uma máquina virtual pode levar minutos para iniciar e ocupar gigabytes, um contêiner inicia em segundos e consome dezenas a centenas de megabytes.

Figura 4: Arquitetura comparada: máquinas virtuais (esquerda) e contêineres Docker (direita)



Fonte: Elaborado pelo autor com base em Salvador (2019b).

O modelo apoia-se em três artefatos (Docker, Inc., 2026): o *Dockerfile*, arquivo de texto que descreve como construir a imagem (base, dependências, código, inicialização); a imagem, resultado somente leitura organizado em camadas, versionável e armazenável em registros como o Docker Hub ou o Amazon ECR; e o contêiner, instância em execução de uma imagem, com seu próprio espaço de processos, rede e armazenamento volátil. A composição de múltiplos contêineres costuma ser descrita em `docker-compose.yml`, e, em produção, orquestradores

como o Kubernetes ou serviços gerenciados como o Amazon ECS provisionam, monitoram, reiniciam e escalam as instâncias.

As vantagens são diretas para sistemas baseados em microsserviços: reprodutibilidade do ambiente (o mesmo Dockerfile gera a mesma imagem em desenvolvimento e em produção), isolamento de dependências (serviços com versões diferentes de uma biblioteca convivem sem conflito) e portabilidade (a imagem roda em qualquer ambiente com Docker Engine). Por essas razões, os contêineres tornaram-se a base de implantação das arquiteturas orientadas a microsserviços para a Indústria 4.0 (Bigheti, 2020).

## 2.6 Computação em Nuvem

O termo *Cloud Computing*, ou Computação em Nuvem, surgiu em 2006 e consolidou-se como um paradigma em que dados e aplicações são movidos para grandes centros de armazenamento e a infraestrutura é distribuída como serviços acessíveis pela internet (Henriques da Silva, 2010). A “nuvem” esconde os recursos físicos sob uma interface padronizada, em parentesco com a *Utility Computing*: a capacidade computacional torna-se uma utilidade tarifada pelo consumo (Pedrosa; Nogueira, 2011). Para ser considerada nuvem, uma plataforma deve reunir cinco características: *self-service* sob demanda, amplo acesso à rede, *pooling* de recursos multi-inquilino, elasticidade rápida e serviço medido (Henriques da Silva, 2010; Pedrosa; Nogueira, 2011).

A arquitetura organiza-se em três camadas de serviço (Figura ??): Infraestrutura como Serviço (IaaS), com recursos virtualizados de hardware; Plataforma como Serviço (PaaS), com ambientes prontos de desenvolvimento; e Software como Serviço (SaaS), com aplicações completas entregues por portais web (Pedrosa; Nogueira, 2011). Quanto à implantação, há as nuvens pública, privada, comunitária e híbrida, esta última capaz de absorver picos de demanda com recursos públicos (Henriques da Silva, 2010).

Entre as vantagens estão a mobilidade, o pagamento por uso, a escalabilidade e a confiabilidade dos grandes provedores; entre os desafios, a segurança, a interoperabilidade entre provedores e a disponibilidade (Henriques da Silva, 2010; Pedrosa; Nogueira, 2011). No contexto da Indústria 4.0, a nuvem é a infraestrutura que viabiliza a integração de produtos, máquinas, sensores e serviços, reposicionando o SCADA, antes confinado a salas de controle locais, como nó de uma rede distribuída de serviços (Bigheti, 2020; Pisching, 2017). Entre os provedores comerciais, a Amazon Web Services (AWS) é atualmente o maior, oferecendo serviços em todas as camadas do modelo sob precificação *pay-per-use*.

## 2.7 Next.js

O Next.js é um *framework* para a construção de aplicações web, criado sobre o React, uma biblioteca JavaScript desenvolvida pela Meta para montar interfaces a partir de componentes, isto é, pequenos blocos reutilizáveis de tela, como botões, formulários e gráficos, que se combinam para formar páginas inteiras. O React, sozinho, monta a interface diretamente no navegador do usuário: envia um arquivo quase vazio acompanhado de um programa em JavaScript que desenha a tela depois que a página carrega. Esse modelo, chamado de renderização no cliente (*client-side rendering*, CSR), funciona bem, mas tem desvantagens, a primeira carga pode ser lenta e os mecanismos de busca têm dificuldade para ler o conteúdo (Vercel, Inc., 2026).

O Next.js surge justamente para contornar essas limitações, oferecendo outras formas de gerar as páginas. Na renderização no servidor (*server-side rendering*, SSR), a página já é

montada pronta no servidor e enviada ao navegador, o que acelera a exibição e facilita a leitura pelos buscadores. Na geração estática (*static site generation*, SSG), a página é montada uma única vez, no momento em que o sistema é publicado, e a mesma versão pronta é entregue a todos os usuários, garantindo o melhor desempenho para conteúdos que mudam pouco. Uma vantagem importante é que o Next.js permite escolher a estratégia mais adequada para cada tela, em vez de impor uma só a toda a aplicação (Vercel, Inc., 2026).

Além das opções de renderização, o Next.js simplifica tarefas comuns do desenvolvimento. O roteamento, isto é, a associação entre os endereços (URLs) e as páginas, baseia-se na própria organização dos arquivos do projeto: cada arquivo criado em uma pasta específica vira automaticamente um endereço da aplicação, sem necessidade de configuração manual. O *framework* ainda traz, de fábrica, recursos que melhoram o desempenho, como carregar apenas o código necessário a cada página, preparar de antemão as páginas que o usuário provavelmente acessará, otimizar imagens automaticamente e oferecer suporte nativo ao TypeScript, uma versão do JavaScript com verificação de tipos que ajuda a evitar erros durante o desenvolvimento (Vercel, Inc., 2026).

## 2.8 HTTP, API REST e JSON

O protocolo HTTP (*Hypertext Transfer Protocol*) é a base da comunicação na web e, por extensão, a base da comunicação entre os serviços de qualquer sistema distribuído moderno. Trata-se de um protocolo cliente-servidor sem estado (*stateless*), em que cada requisição contém todas as informações necessárias para ser processada, sem dependência de requisições anteriores. Uma requisição HTTP é composta por método (GET, POST, PUT, PATCH, DELETE, entre outros), caminho do recurso (URL), cabeçalhos (*headers*) e, opcionalmente, corpo (*body*). A resposta contém código de status (200, 201, 400, 401, 404, 500 etc.), cabeçalhos e corpo.

Sobre o HTTP, o padrão REST (*Representational State Transfer*), proposto por Roy Fielding em sua tese de doutorado em 2000, estabelece um conjunto de restrições arquiteturais para projetar APIs web bem comportadas. Os princípios centrais incluem: identificação clara de recursos por URL (`/equipamentos/42/tags`); uso dos métodos HTTP de forma semântica (GET para consulta, POST para criação, PUT para atualização completa, PATCH para atualização parcial, DELETE para remoção); representação independente da forma interna (frequentemente JSON na comunicação cliente-servidor moderna); ausência de estado de sessão no servidor; e estrutura hierárquica de recursos que reflete o domínio do problema. APIs REST bem desenhadas são previsíveis, navegáveis e composíveis, propriedades alinhadas com o paradigma de orientação a serviços discutido por Bangemann *et al.* (2014) no contexto de automação industrial.

O formato JSON (*JavaScript Object Notation*) é a representação de dados padrão de fato para APIs REST modernas. Trata-se de uma sintaxe textual derivada do JavaScript, simples de ler e escrever para humanos e fácil de gerar e parsear para máquinas. Suporta tipos primitivos (*string*, número, booleano, nulo) e estruturas compostas (objeto e *array*), o que se mostrou suficiente para a grande maioria dos cenários de troca de dados. JSON substituiu, em larga medida, o XML em comunicações web devido à sua menor verbosidade e ao acoplamento natural com o JavaScript dos navegadores. Adicionalmente, esquemas JSON (*JSON Schema*) permitem descrever formalmente o formato esperado de mensagens, oferecendo validação e geração de documentação automaticamente.

Em sistemas industriais conectados, a comunicação por API REST sobre HTTP é hoje uma alternativa cada vez mais comum ao acoplamento direto ao protocolo Modbus. Como observa Bangemann *et al.* (2014), a exposição de funcionalidades de dispositivos de campo como serviços

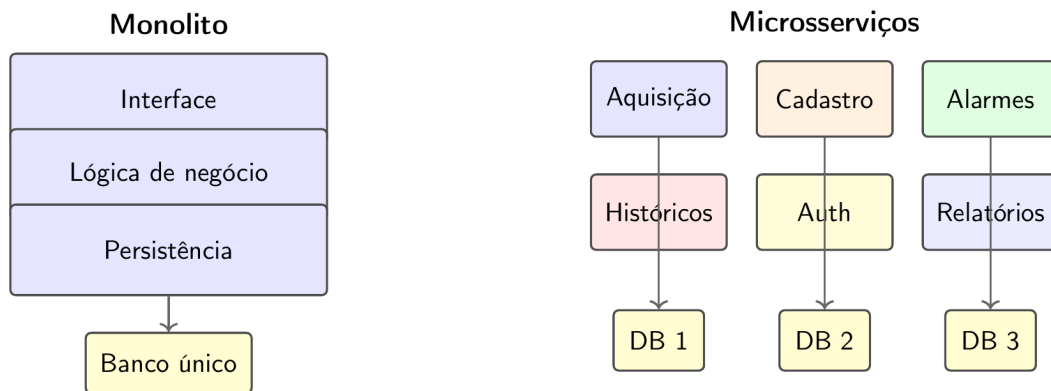
web (*Web Services*) é um dos vetores que viabilizam o achatamento da pirâmide ISA-95.

## 2.9 Microsserviços

O paradigma de microsserviços é um estilo arquitetural em que a aplicação é estruturada como um conjunto de serviços pequenos, autônomos e independentemente implantáveis, cada um responsável por um domínio bem delimitado e comunicando-se por protocolos leves, tipicamente APIs HTTP/REST ou mensageria assíncrona. Popularizou-se a partir de 2014, herdeiro da *Service-Oriented Architecture* (SOA) discutida por Bangemann *et al.* (2014) no contexto industrial; a diferença está na granularidade e na autonomia: em vez de serviços grandes orquestrados por um barramento central, adotam-se serviços pequenos, com bancos próprios e ciclos de vida independentes.

A motivação vem das limitações do monolito, em que todas as responsabilidades vivem em um único processo, versionado e implantado em conjunto e escalado como um todo: à medida que cresce, a base de código fica difícil de evoluir, o tempo de implantação aumenta, uma falha pode comprometer o sistema inteiro e a escalabilidade aplica-se ao todo, não à parte realmente sobrecarregada (Figura 5).

Figura 5: Aplicação monolítica (esquerda) versus arquitetura de microsserviços (direita)



Fonte: Elaborado pelo autor com base em Bigheti (2020).

As vantagens centrais, segundo Bigheti (2020), são a implantação independente (cada serviço evolui sem afetar os demais), a escalabilidade granular (replica-se apenas o gargalo) e a heterogeneidade tecnológica (cada serviço usa a linguagem e o banco mais adequados). No contexto da Indústria 4.0, o autor defende que esses serviços alinham-se aos princípios de modularidade, descentralização e orientação a serviços da própria I4.0 (Pisching, 2017). Há, porém, custos: a complexidade migra para o ecossistema operacional (dezenas de serviços, réplicas, observabilidade), a consistência passa a depender de mecanismos eventuais, e padrões como *API Gateway*, *Service Registry*, *Circuit Breaker* e *Distributed Tracing* tornam-se essenciais. Uma diretriz comum de decomposição é o princípio de *bounded contexts* do *Domain-Driven Design*, em que cada serviço corresponde a um domínio bem delimitado do problema.

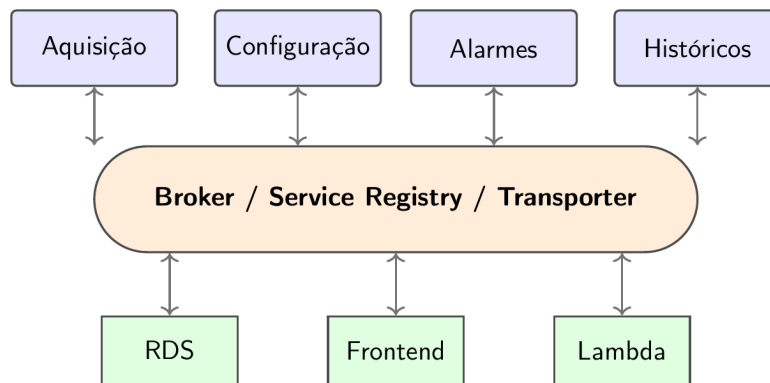
## 2.10 Moleculer

O Moleculer é um *framework* de microsserviços para Node.js, voltado a sistemas distribuídos compostos por serviços pequenos e independentemente implantáveis (MoleculerJS, 2026).

Posiciona-se como alternativa pragmática a soluções mais pesadas, como o Spring Cloud no ecossistema Java, reunindo descoberta de serviços, balanceamento de carga, tolerância a falhas e diversos transportes (TCP, NATS, Redis, MQTT, Kafka) em uma única biblioteca leve.

No modelo do Moleculer, cada serviço define *actions* (ações invocáveis remotamente, como métodos de uma API) e *events* (mensagens publicadas para outros serviços), e registra-se em um *broker* que faz a descoberta automática dos nós e roteia chamadas e eventos para a instância adequada, com balanceamento entre réplicas (Figura 6). Diante de falhas, o broker redireciona a chamada para outra instância, aplica *circuit breakers* contra cascatas de falha ou retorna respostas *fallback* (MoleculerJS, 2026).

Figura 6: Arquitetura de microsserviços baseada em barramento de eventos (broker Moleculer)



Fonte: Elaborado pelo autor com base em Bigheti (2020).

O *framework* suporta múltiplos transportes: em desenvolvimento, TCP direto; em produção, barramentos resilientes como NATS, Redis Pub/Sub ou MQTT, com persistência opcional e entrega garantida. Transportes como o NATS oferecem ainda *streams* persistentes, úteis para reprocessar eventos quando um serviço se recupera de falha, recurso alinhado às arquiteturas de microsserviços para a Indústria 4.0 (Bigheti, 2020).

## 2.11 Protocolo Modbus TCP e Modbus RTU

O Modbus é um dos protocolos de comunicação mais difundidos na automação industrial. Segue o modelo de requisição e resposta entre um cliente, que faz os pedidos, e um ou mais servidores, que são os dispositivos de campo, tradicionalmente chamados de *master* e *slaves*. Em um sistema de supervisão, o cliente envia pedidos para ler ou escrever valores no dispositivo, e este responde com os dados solicitados ou com um código de erro quando a operação não é permitida (Modbus Organization, 2012a).

Os dados de um dispositivo Modbus ficam organizados em quatro áreas de memória: *coils* e *input coils*, para estados do tipo booleano, e *holding registers* e *input registers*, para valores numéricos de 16 bits. Cada operação é identificada por um código de função, havendo funções específicas para ler ou escrever em cada uma dessas áreas (Modbus Organization, 2012a).

Para que um valor de processo seja lido corretamente, é preciso configurar, para cada ponto, alguns parâmetros definidos pelo próprio protocolo, que indicam onde o dado se encontra no dispositivo: o endereço do dispositivo, que identifica o equipamento, o código de função, que define a área de memória e a operação, o endereço inicial e a quantidade de registradores a serem lidos. Esses parâmetros formam a base da configuração de uma *tag* Modbus e permitem que

um mesmo software de aquisição se comunique com medidores e controladores de fabricantes diferentes apenas alterando a configuração, sem mudar o código.

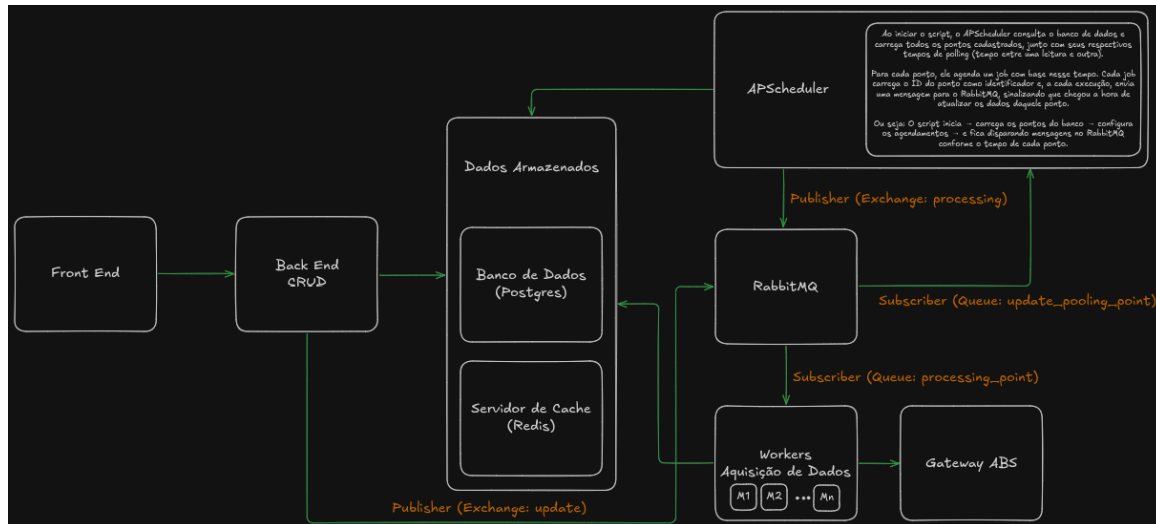
Além desses parâmetros do protocolo, é preciso atenção especial à forma como o dado chega, pois é nesse ponto que surgem as maiores diferenças entre fabricantes. Cada registrador Modbus tem apenas 16 bits, mas muitos valores ocupam mais espaço: um inteiro de 32 bits ou um número de ponto flutuante usam dois registradores, e um valor de 64 bits, quatro. É necessário, portanto, configurar corretamente o tipo de dado bruto e a ordem em que esses registradores e seus bytes são combinados, a chamada ordem de bytes e de palavras, que varia de um fabricante para outro e, se interpretada de forma errada, produz um valor incorreto. Por fim, quando aplicável, configuram-se os fatores de escala que convertem o valor bruto em uma grandeza de engenharia.

O Modbus possui duas variantes principais, que se diferenciam pelo meio de transporte. O *Modbus TCP* trafega sobre redes TCP/IP, identificando o dispositivo por um endereço IP e um *Unit ID*, e é comum em equipamentos com interface Ethernet. O *Modbus RTU* é a forma usada em redes seriais, normalmente sobre o padrão RS-485, em que os dispositivos são identificados pelo endereço de escravo (Modbus Organization, 2012b).

### 3 Funcionamento

O funcionamento do sistema organiza-se em camadas, conforme ilustrado na Figura 7. Essa divisão separa as responsabilidades entre a interface de supervisão, os serviços que rodam na nuvem, o armazenamento das informações e a comunicação com os equipamentos instalados em campo.

Figura 7: Arquitetura do Sistema



Fonte: Autor.

A Figura 7 detalha a arquitetura completa do sistema. A aplicação web é feita em Next.js e reúne, em um único projeto, a interface de supervisão usada pelos operadores e a camada de *backend*, formada pelas rotas de API (*API routes*) que tratam do cadastro de equipamentos, da configuração de *tags*, da consulta de históricos e do envio de comandos. A camada de dados junta o banco relacional PostgreSQL, gerenciado pelo Amazon RDS, e o cache Redis, no Amazon ElastiCache; juntos, eles guardam o histórico de longo prazo e mantêm o valor atual de cada *tag* sempre acessível e com baixa latência. O agendamento fica por conta do BullMQ, uma biblioteca de filas que se apoia no próprio Redis: ao iniciar, ele carrega do banco todos os pontos cadastrados com seus tempos de *polling* e cria, para cada ponto, um *job* que se repete no intervalo definido; a cada disparo, envia uma mensagem com o identificador do ponto, avisando que chegou a hora de atualizar aquele dado. A troca de mensagens entre os serviços é feita pelo Moleculer, que usa o Redis como meio de transporte (*transporter*): ele recebe os pedidos de leitura no canal **processing** e os repassa aos *workers* de aquisição (M1, M2, ..., Mn). Esses *workers* consomem o canal **processing\_point**, abrem a conexão com o dispositivo de campo, fazem a leitura Modbus correspondente e devolvem o resultado ao barramento; o Moleculer, então, publica no canal **update** a leitura já tratada, para que o banco grave o histórico e o cache atualize o valor corrente.

#### 3.1 Validações dos Requisitos do SCADA

As subseções a seguir mostram, item por item, como o sistema atende aos requisitos clássicos do SCADA descritos por Zanghi (2019). Cada validação aponta de que forma a arquitetura da Figura 7, descrita acima, cumpre os seis pilares definidos pelo autor: aquisição e registro de

dados, sincronismo de tempo, níveis de acesso, comando e controle, escalabilidade e redundância.

### 3.1.1 Aquisição e Registro de Dados

Na aquisição e no registro de dados, o sistema atende a tudo o que Zanghi (2019) descreve. A leitura é feita pelos workers, que conversam diretamente com os dispositivos de campo por Modbus e entregam os valores em formato digital ao supervisor. Cada ponto é cadastrado como *tag* no banco de dados e classificado como digital (entrada DI, saída DO, memória DM), para representar estados de liga/desliga, como o de motores e a abertura/fechamento de válvulas, ou como analógico (entrada AI, saída AO, memória AM), para grandezas de vários bits, como pressão, vazão, nível e corrente. O registro contínuo das leituras no PostgreSQL alimenta as análises posteriores, em gráficos de tendência e em relatórios de eventos digitais, enquanto o cache Redis mantém o valor atual de cada ponto pronto para desenhar as telas de supervisão, sem sobrecarregar o banco principal a cada atualização.

### 3.1.2 Sincronismo de Tempo

O sincronismo de tempo exigido pelo SCADA vem da combinação entre o relógio do BullMQ, ajustado por NTP com a infraestrutura da AWS, e o carimbo de tempo que os workers aplicam no instante exato em que recebem a resposta Modbus do dispositivo de campo. Cada mensagem que passa pelo barramento Molecular leva esse carimbo, que segue inalterado por toda a cadeia: do worker para o canal `update`, do canal para o banco e para o cache, do banco para o backend e do backend para a tela do operador. Assim, alarmes e eventos digitais captados em pontos diferentes do sistema recebem marcações de tempo coerentes na casa dos milissegundos, o que atende à exigência de uma cronologia precisa para a análise *a posteriori* de ocorrências e faltas, sobretudo em processos rápidos. Cada valor adquirido fica, portanto, ligado ao carimbo de tempo do servidor e, quando o equipamento oferece, a referências de tempo adicionais, e segue para o histórico e para a atualização das telas; desse modo, a aquisição fecha o laço entre o processo físico e a sua representação digital, preservando a rastreabilidade e as condições de auditoria da operação.

### 3.1.3 Níveis de Acesso

Nos níveis de acesso, o sistema troca a hierarquia fixa de subestação por níveis definidos por planta. Cada usuário faz login no frontend, em Next.js, e recebe uma sessão ligada às instalações sob sua responsabilidade; a camada de backend da própria aplicação Next.js funciona como ponto de autorização e verifica, a cada chamada às rotas de API, se o cargo daquele usuário permite consultar dados, configurar *tags* ou enviar comandos para a planta em questão. Dessa forma, a supervisão e o controle ficam separados conforme o cargo de cada operador, em linha com o controle de acesso baseado em cargos e com o princípio do menor privilégio (Stouffer *et al.*, 2015). Como a solução é multiplanta e compartilha a mesma infraestrutura na nuvem, esse isolamento de permissões por instalação também é uma medida de segurança, pois impede que usuários de um cliente acessem ou comandem pontos de outro (Wali; Alshehry, 2024).

### 3.1.4 Comando e Controle

O comando e o controle previstos pelo SCADA são implementados nas três variantes descritas por Zanghi (2019). O telecomando instantâneo, em que o operador, pela tela, muda na

hora o estado de um equipamento (liga uma bomba, abre uma válvula, aciona um disjuntor), vira uma escrita Modbus feita pelo worker correspondente. O telecontrole, em que o operador define um *set point* para regular ao longo do tempo a resposta de um processo, segue o mesmo caminho, mas atualiza um *holding register* específico. Já o telecomando ou telecontrole programado, em que as ações são agendadas conforme cenários definidos antes, fica guardado como configuração no banco e é disparado pelo BullMQ na hora marcada, com o instante de execução registrado. As quatro fases do processo (seleção, verificação, confirmação e execução) ficam divididas entre a tela (seleção e confirmação explícita por uma janela de aviso), o backend da aplicação Next.js (verificação de intertravamentos e condições de segurança) e o worker (execução da escrita Modbus); após executar, o próprio worker faz uma leitura de verificação para confirmar que o dispositivo de campo aceitou o pedido e, se algo falhar, gera o alarme correspondente para avisar o operador da inconsistência.

### 3.1.5 Escalabilidade

A escalabilidade é atendida em três frentes que se complementam. Na expansão do cadastro, é possível incluir novos pontos e novas telas com o sistema em produção, sem mexer no código: o worker é parametrizado pelo banco e o frontend monta as telas de forma dinâmica conforme a configuração; o BullMQ percebe a inclusão, recarrega a lista de pontos e ajusta seus *jobs*. Na capacidade de processamento, os workers (M1, M2, . . . , Mn) podem ser replicados na horizontal para dividir a carga: o Moleculer faz o balanceamento automático entre as instâncias que consomem o canal `processing_point`, e o Amazon ECS cria novas réplicas dos contêineres Docker conforme o uso. Na infraestrutura de dados, os serviços gerenciados Amazon RDS e ElastiCache crescem na vertical sob demanda. Esse desenho elimina o gargalo de licenciamento por número de pontos, comum nos supervisórios comerciais, e permite acompanhar o crescimento das instalações sem refazer o orçamento da solução, ainda que a capacidade dos elementos de rede e do barramento Moleculer precise ser pensada desde o projeto.

### 3.1.6 Redundância

A redundância acompanha a arquitetura em todas as camadas críticas, seguindo as recomendações de Zanghi (2019) e Salvador (2019a). Os workers de aquisição funcionam como uma frota em que cada instância pega trabalho de uma mesma fila, gerenciada pelo Moleculer, de modo que a perda de um worker é absorvida pelos demais, sem perder leituras pendentes. O banco em RDS é configurado com uma réplica em outra zona de disponibilidade, com troca automática (*failover*) caso o nó principal falhe e com *snapshots* programados para recuperação ponto a ponto. O cache Redis opera em modo *cluster*, com réplicas espelhadas; como ele também sustenta o transporte do Moleculer e as filas do BullMQ, essa redundância protege, de uma só vez, o cache, o barramento e o agendamento. A aplicação Next.js (frontend e backend) roda em vários contêineres atrás de um balanceador de carga, e o próprio barramento Moleculer pode ser configurado em *cluster* para tolerar falhas no transporte. Esse conjunto de medidas alcança níveis de confiabilidade compatíveis com a criticidade das aplicações industriais mais sensíveis.

## 3.2 Servidores

O sistema se divide em seis componentes lógicos de servidor, cada um com uma responsabilidade bem delimitada e um ciclo de vida próprio, em linha com o paradigma de microsserviços

defendido por Bigheti (2020) para sistemas industriais orientados a serviços. Todos os componentes são empacotados como contêineres Docker e orquestrados pelo Amazon ECS, o que permite implantar cada um de forma independente, escalar de modo granular e isolar dependências. Cada um dos seis servidores é descrito nas subseções a seguir quanto à tecnologia escolhida, à sua responsabilidade no sistema e às interfaces que mantém com os demais.

### 3.2.1 Aplicação Next.js (Frontend e Backend)

A aplicação web é desenvolvida em Next.js e concentra, em um único projeto, tanto a interface de supervisão quanto o *backend* de dados, este último na forma de rotas de API (*API routes*). Do lado da interface, as telas incluem a visualização de pontos ao vivo, os *dashboards* de tendência histórica, o cadastro de equipamentos e *tags*, os painéis de alarmes ativos e os formulários administrativos. As telas de operação em tempo real são desenhadas no próprio navegador (renderização no cliente) e atualizadas aos poucos por WebSocket, o que garante pouca demora entre a leitura no campo e a apresentação ao operador; já as telas de cadastro e os relatórios são gerados no servidor (SSR), aproveitando o pré-processamento e a checagem de credenciais.

Do lado do backend, as rotas de API expõem as operações consumidas pela interface: criação, leitura, atualização e remoção do cadastro de clientes, unidades, equipamentos e *tags*; consultas ao histórico com filtros de tempo, equipamento e ponto; gestão de usuários, cargos e permissões; e o envio de comandos pelo operador. Cada requisição é conferida contra um esquema declarativo (JSON Schema), autenticada por um *token* JWT guardado no Redis e registrada em *log* estruturado. Quando uma operação precisa interagir com o campo, como em uma escrita Modbus, o backend não fala direto com os workers: ele publica um comando no barramento Molecular, mantendo a camada de aplicação desacoplada da camada de aquisição. Toda a comunicação entre o navegador e a aplicação acontece por REST/JSON e por WebSocket; o acesso ao banco, ao cache e ao barramento de eventos fica restrito à camada de backend, nunca partindo diretamente do cliente.

### 3.2.2 Banco de Dados

O servidor de banco de dados é uma instância PostgreSQL gerenciada pelo Amazon RDS, responsável por guardar, por longo prazo, todo o estado relevante do sistema: o cadastro hierárquico (cliente, unidade operacional, equipamento, *tag*), o histórico de leituras com carimbo de tempo, os eventos de alarme, os registros de auditoria, a configuração de usuários e os parâmetros de comunicação. O cadastro organiza a aquisição em níveis hierárquicos e guarda, para cada ponto, o mapeamento Modbus necessário: o endereço da unidade (*Unit ID*), o código de função (*holding registers*, *input registers*, *coils* ou *discrete inputs*), o endereço inicial e a quantidade de elementos, o tipo de dado bruto (inteiro de 16 ou 32 bits, *float* ou várias palavras) e a ordem de bytes e *words*, além dos fatores de escala e do *offset* para converter o valor em unidade física, dos limites de engenharia e da classificação do ponto (leitura periódica, leitura sob demanda ou escrita supervisionada); com esse modelo, o mesmo serviço de aquisição atende processos diferentes apenas por configuração, sem mudar o código a cada novo medidor ou CLP. O esquema relacional usa chaves estrangeiras para manter a integridade referencial; as tabelas de histórico são particionadas por intervalo de tempo, para não virarem tabelas gigantes, e índices nos campos *tag\_id* e *timestamp* aceleram as consultas mais frequentes. Em produção, o banco usa uma réplica em outra zona de disponibilidade, com *failover* automático caso o nó principal falhe e *snapshots* programados para recuperação ponto a ponto, conforme recomenda Salvador (2019a).

### 3.2.3 Cache

O servidor de cache é uma instância Redis fornecida pelo Amazon ElastiCache e cumpre vários papéis ao mesmo tempo. Como cache de leituras frequentes, mantém em memória o valor atual de cada *tag* ativa, acessado pelas telas de supervisão em tempo real sem onerar o banco principal a cada atualização. Como base do barramento de mensagens, é o Redis que serve de meio de transporte (*transporter*) para o Moleculer e de suporte para as filas de agendamento do BullMQ, concentrando em um único componente o transporte de eventos e o apoio às filas. Como armazenamento temporário, guarda os *tokens* de autenticação, os contadores que limitam o número de requisições por usuário (*rate limiting*) e as travas distribuídas que evitam o processamento duplicado de comandos críticos. O cluster opera com réplicas espelhadas em zonas distintas e com expiração automática (TTL) das chaves temporárias.

### 3.2.4 BullMQ

O servidor de agendamento executa a biblioteca BullMQ em um contêiner dedicado e tem a função de decidir o instante em que cada ponto deve ser lido. Como o BullMQ se apoia no Redis para guardar e distribuir suas filas, o agendamento aproveita a mesma infraestrutura de cache já presente no sistema. Ao iniciar, o serviço consulta o banco, carrega todos os pontos cadastrados com seus tempos de *polling* (o intervalo entre uma leitura e a próxima) e cria, para cada ponto, um *job* repetível baseado nesse tempo. As cadências variam conforme a grandeza: de minutos, para variáveis lentas, como o nível de um reservatório grande, a segundos, para indicadores críticos, como a pressão de sucção de uma bomba; o agendador ainda evita sobrepor ciclos longos de um mesmo equipamento quando a leitura anterior ainda não terminou. A cada disparo, o *job* carrega o identificador do ponto e publica uma mensagem no barramento Moleculer, avisando os workers de que chegou a hora de atualizar aquele dado. O servidor reage a mudanças no cadastro (inclusão, remoção ou troca de periodicidade) recarregando os *jobs* afetados, sem precisar reiniciar; janelas de manutenção e leituras prioritárias também são tratadas aqui, separando a decisão de “quando ler” da execução de “como ler” e coordenando a telemetria periódica, voltada à tendência, com as leituras sob demanda disparadas pela interface quando o operador abre uma tela detalhada.

### 3.2.5 Barramento Moleculer

O servidor de barramento de mensagens é uma instância do Moleculer que coordena a comunicação assíncrona entre os demais componentes, usando o Redis como meio de transporte (*transporter*). Três canais principais organizam o tráfego: o canal **processing** recebe os pedidos de leitura vindos do BullMQ e do backend (no caso das leituras sob demanda); o canal **processing\_point** é consumido pelos workers de aquisição em modo de balanceamento de carga; e o canal **update** leva o resultado das leituras de volta para o banco e para o cache, além de servir de gatilho para os serviços de alarme e de notificação. O Moleculer ainda oferece descoberta automática de serviços, balanceamento entre instâncias replicadas, disjuntores (*circuit breakers*) que isolam falhas em cascata e rastreamento distribuído (*distributed tracing*), que permite acompanhar uma requisição de ponta a ponta quando ela passa por vários serviços.

### 3.2.6 Workers de Aquisição

Os workers de aquisição (M1, M2, ..., Mn) formam a frota responsável pela comunicação efetiva com o campo. Cada worker é uma instância idêntica de um contêiner Docker, replicável

na horizontal conforme a carga, e mantém um conjunto de conexões TCP ativas com os dispositivos de campo de cada cliente. Ao consumir uma mensagem do canal `processing_point`, o worker percorre o ciclo de ler a configuração do ponto, montar o telegrama Modbus correspondente, enviá-lo ao dispositivo e validar a resposta, checando a coerência do tamanho, as exceções Modbus (códigos de erro de função), os tempos de resposta e a integridade básica dos dados; em seguida, decodifica o valor bruto, aplicando máscaras e combinações de registradores e a conversão para a unidade de engenharia, e publica o resultado no canal `update`.

Quando a leitura falha, por *timeout*, perda de enlace ou recusa do escravo, o worker registra a ocorrência, mantém o último valor válido com a marca de *desatualizado* e alimenta os mecanismos de alarme de comunicação da supervisão, pois, em um SCADA, a falta de um dado precisa ser tão visível quanto um limite de processo ultrapassado. Para não saturar os enlaces móveis compartilhados, políticas conservadoras de concorrência limitam o número de conexões TCP paralelas contra o mesmo dispositivo e agrupam leituras em blocos contíguos de registradores quando o equipamento permite, reduzindo o vai e volta de mensagens. Os workers são *stateless*, o que permite adicionar ou remover instâncias sem perda de estado; o *timeout*, o número de tentativas e o *backoff* exponencial são parametrizados por equipamento ou por *site*, equilibrando resiliência e carga na rede.

### 3.3 Aquisição de Dados

A camada de aquisição de dados é o subsistema que traduz o estado físico do processo (níveis, vazões, pressões, energia, status de bombas e válvulas, alarmes, contadores etc.) em valores digitais confiáveis, com referência de tempo e com metadados suficientes para serem usados pela supervisão, pelos relatórios operacionais e pelas rotinas de histórico. Na prática, ela vai da montagem das requisições até a entrega dos dados já convertidos para as unidades de engenharia (os valores nas unidades físicas usuais na operação, como metro, m<sup>3</sup>/h, bar ou kW). É comum que o equipamento mostre a grandeza apenas como um número inteiro em um ou mais registradores Modbus; o programa do CLP grava, então, uma representação em ponto fixo (por exemplo, o nível em centímetros com uma casa decimal vem como inteiro que precisa ser dividido por 10, ou a vazão usa divisor 100), e cabe à camada de aquisição aplicar a multiplicação ou divisão, as somas de deslocamento (*offset*) e os demais fatores de escala cadastrados, recompondo o valor real usado na operação, pronto para gravar no banco.

Nesse arranjo, os workers são a fronteira do sistema com o campo: é neles que termina o escopo deste trabalho. A partir do worker, a leitura e a escrita das variáveis de processo usam o protocolo Modbus, e a parte física de conectividade, formada pelos equipamentos de telemetria que levam o sinal até o dispositivo remoto, fica fora do recorte aqui tratado. Cada worker abre a conexão com o dispositivo de campo, troca os quadros Modbus e devolve ao barramento o valor já lido, sem depender de um concentrador específico de fornecedor.

O encadeamento entre o worker e o dispositivo escravo preserva a semântica das funções Modbus (códigos de função, endereços de registradores ou *coils*, dados de resposta), variando apenas o “envelope” de transporte: quadros TCP/IP quando a comunicação é em rede e, no trecho serial, o quadro RTU com verificação por CRC.

## 4 Cronograma

O cronograma de atividades previstas para o desenvolvimento deste Trabalho de Conclusão de Curso é apresentado na Tabela 1, distribuindo as atividades ao longo dos doze meses do ano, organizadas conforme as fases de Projeto de Trabalho de Conclusão, Trabalho de Conclusão e Defesa.

Tabela 1: Cronograma de atividades do TCC.

| ATIVIDADE                                        | MÊS |    |    |    |    |    |    |    |    |    |    |    |
|--------------------------------------------------|-----|----|----|----|----|----|----|----|----|----|----|----|
|                                                  | 01  | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 10 | 11 | 12 |
| Definição do Tema                                | X   |    |    |    |    |    |    |    |    |    |    |    |
| Pesquisa Bibliográfica                           |     | X  | X  | X  |    |    |    | X  | X  | X  |    |    |
| Elaboração do Projeto de Trabalho de Conclusão   |     |    | X  | X  | X  | X  |    |    |    |    |    |    |
| Entregas ao Orientador                           |     |    |    | X  | X  | X  |    | X  | X  | X  | X  |    |
| Apresentação do Projeto de Trabalho de Conclusão |     |    |    |    |    | X  |    |    |    |    |    |    |
| Elaboração do Trabalho de Conclusão              |     |    |    |    |    |    | X  | X  | X  | X  | X  | X  |
| Apresentação do Trabalho de Conclusão            |     |    |    |    |    |    |    |    |    |    |    | X  |
| Defesa do Trabalho de Conclusão                  |     |    |    |    |    |    |    |    |    |    |    | X  |

Fonte: Autor.

## Referências

- ABBAS, Hosny Ahmed. **Efficient Web-Based SCADA System**. 2011. Dissertação (Mestrado em Engenharia Elétrica) – Faculty of Engineering, South Valley University, Aswan.
- BANGEMANN, Thomas *et al.* State of the Art in Industrial Automation. *In*: COLOMBO, Armando W. *et al.* (ed.). **Industrial Cloud-Based Cyber-Physical Systems: The IMC-AESOP Approach**. Cham: Springer, 2014. p. 23–47. ISBN 978-3-319-05623-4. DOI: 10.1007/978-3-319-05624-1\_2.
- BIGHETI, Jeferson André. **Arquitetura de Automação e Controle Orientada a Microserviços para a Indústria 4.0**. 2020. Tese (Doutorado em Engenharia Elétrica) – Faculdade de Engenharia, Universidade Estadual Paulista “Júlio de Mesquita Filho”, Bauru.
- DAMIN, Délio. **Melhores práticas para o desenvolvimento de uma aplicação Elipse E3**. Atualizado em 22 dez. 2022. Elipse Software. 7 jul. 2022. Disponível em: <https://kb.elipse.com.br/melhores-praticas-para-o-desenvolvimento-de-uma-aplicacao-elipse-e3/>. Acesso em: 20 maio 2026.
- DOCKER, INC. **Docker Documentation**. Docker, Inc. Disponível em: <https://docs.docker.com/>. Acesso em: 14 jun. 2026.
- HENRIQUES DA SILVA, Fabrício Rodrigues. **Um estudo sobre os benefícios e os riscos de segurança na utilização de Cloud Computing**. 2010. Trabalho de Conclusão de Curso (Graduação em Ciência da Computação) – Centro Universitário Augusto Motta, Rio de Janeiro.
- KHALID, Wajahat *et al.* Open-Source Internet of Things-Based Supervisory Control and Data Acquisition System for Photovoltaic Monitoring and Control Using HTTP and TCP/IP Protocols. **Energies**, v. 17, n. 16, p. 4083, 2024. DOI: 10.3390/en17164083.
- LASI, Heiner *et al.* Industry 4.0. **Business & Information Systems Engineering**, v. 6, n. 4, p. 239–242, 2014. DOI: 10.1007/s12599-014-0334-4.
- MODBUS ORGANIZATION. **Modbus Messaging on TCP/IP Implementation Guide**. Versão 1.0b. Hopkinton, MA, 2012. Disponível em: [https://www.modbus.org/docs/Modbus\\_Messaging\\_Implementation\\_Guide\\_V1\\_0b.pdf](https://www.modbus.org/docs/Modbus_Messaging_Implementation_Guide_V1_0b.pdf). Acesso em: 15 maio 2026.
- MODBUS ORGANIZATION. **Modbus over Serial Line Specification and Implementation Guide**. Versão 1.02. Hopkinton, MA, 2012. Disponível em: [https://www.modbus.org/docs/Modbus\\_over\\_serial\\_line\\_V1\\_02.pdf](https://www.modbus.org/docs/Modbus_over_serial_line_V1_02.pdf). Acesso em: 15 maio 2026.
- MOLECULERJS. **Molecular Documentation**. MoleculerJS. Disponível em: <https://moleculer.services/docs/0.14/>. Acesso em: 14 jun. 2026.
- MONOSTORI, L. *et al.* Cyber-physical systems in manufacturing. **CIRP Annals**, v. 65, n. 2, p. 621–641, 2016. DOI: 10.1016/j.cirp.2016.06.005.

PEDROSA, Paulo Henrique C.; NOGUEIRA, Tiago. Computação em Nuvem. Trabalho acadêmico (disciplina G04). [S. l.], 2011.

PHUYAL, Sudip *et al.* Design and Implementation of Cost Efficient SCADA System for Industrial Automation. **International Journal of Engineering and Manufacturing**, v. 10, n. 2, p. 15–28, 2020. DOI: 10.5815/ijem.2020.02.02.

PISCHING, Marcos André. **Arquitetura para descoberta de equipamentos em processos de manufatura com foco na indústria 4.0**. 2017. Tese (Doutorado em Engenharia de Controle e Automação Mecânica) – Escola Politécnica, Universidade de São Paulo, São Paulo.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL Documentation**. PostgreSQL Global Development Group. Disponível em: <https://www.postgresql.org/docs/current/>. Acesso em: 14 jun. 2026.

REDIS LTD. **Redis Documentation**. Redis Ltd. Disponível em: <https://redis.io/docs/latest/>. Acesso em: 14 jun. 2026.

SALVADOR, Marcelo Barbosa. **Aquisição de Dados Críticos e Redundância em Sistemas SCADA**. Elipse Software. 25 mar. 2019. Disponível em: <https://kb.elipse.com.br/aquisicao-de-dados-criticos-e-redundancia-em-sistemas-scada/>. Acesso em: 20 maio 2026.

SALVADOR, Marcelo Barbosa. **Guia de Virtualização Elipse Software**. Atualizado em 4 set. 2024. Elipse Software. 25 mar. 2019. Disponível em: <https://kb.elipse.com.br/guia-de-virtualizacao-elipse-software/>. Acesso em: 20 maio 2026.

STANKOVIC, John A. Research Directions for the Internet of Things. **IEEE Internet of Things Journal**, v. 1, n. 1, p. 3–9, 2014. DOI: 10.1109/JIOT.2014.2312291.

STOUFFER, Keith *et al.* **Guide to Industrial Control Systems (ICS) Security**. Gaithersburg, MD, maio 2015. DOI: 10.6028/NIST.SP.800-82r2. Disponível em: <https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-82r2.pdf>. Acesso em: 13 jun. 2026.

VERCEL, INC. **Next.js Documentation**. Vercel, Inc. Disponível em: <https://nextjs.org/docs>. Acesso em: 14 jun. 2026.

WALI, Arwa; ALSHEHRY, Fatimah. A Survey of Security Challenges in Cloud-Based SCADA Systems. **Computers**, v. 13, n. 4, p. 97, 2024. DOI: 10.3390/computers13040097.

YADAV, Geeta; PAUL, Kolin. **Architecture and Security of SCADA Systems: A Review**. arXiv:2001.02925 [cs.CR]. arXiv. 9 jan. 2020. Disponível em: <https://arxiv.org/abs/2001.02925>. Acesso em: 13 jun. 2026.

ZANGHI, Eric. **Sistemas SCADA: Conceitos**. Porto, Portugal, 2019. Série Proteção e Comunicação de Sistemas Elétricos de Potência (PROTCOM).